

Мандатное разграничение прав в Linux (SELinux)

Отчёт по лабораторной работе №6

Алексей Прядко

Содержание

1. Цель работы

Развить навыки администрирования ОС Linux. Получить первое практическое знакомство с технологией SELinux. Проверить работу SELinux на практике совместно с веб-сервером Apache.

2. Задание

1. Подготовить лабораторный стенд: установить и запустить веб-сервер Apache, отключить фаервол, убедиться, что SELinux работает в режиме enforcing с политикой targeted.
2. Определить SELinux-контекст процессов httpd, изучить булевы переключатели.
3. Проанализировать метки файлов в /var/www, создать тестовый HTML-файл и проверить доступ к нему.
4. Изменить контекст файла на неразрешённый для httpd, продемонстрировать блокировку доступа и проанализировать логи SELinux.
5. Вернуть разрешённый контекст и восстановить доступ.
6. Перенести Apache на порт 81, исследовать управление сетевыми портами через SELinux, проверить доступ.
7. Вернуть стандартный порт 80, удалить временные объекты.
8. Оформить отчёт.

3. Теоретическое введение

SELinux (Security-Enhanced Linux) – реализация мандатного контроля доступа (MAC), встроенная в ядро Linux. В отличие от дискреционного управления, SELinux ограничивает действия субъектов (процессов) над объектами (файлами, портами) на основе централизованной политики. Основные компоненты:

- **Контекст безопасности** – метка вида `пользователь:роль:тип:уровень`, присваиваемая каждому субъекту и объекту. Тип (type) является ключевым элементом политики.
- **Домен процесса** – тип, определяющий набор разрешённых действий.
- **Политика targeted** – в современных дистрибутивах по умолчанию ограничиваются только наиболее уязвимые службы (например, `httpd`, `sshd`), остальные процессы работают в неограниченном домене `unconfined_t`.
- **Режимы работы**: `enforcing` – запрещающий с аудитом, `permissive` – только аудит без блокировки, `disabled`.
- Управление портами: SELinux контролирует, к каким портам могут привязываться сервисы, с помощью типа порта (например, `http_port_t`).
- Утилиты: `ps -Z`, `ls -Z` – просмотр контекстов; `chcon` – временное изменение контекста; `restorecon` – восстановление контекста по правилам; `semanage` – управление политикой; `getsebool` – просмотр логических переключателей; `sealert` – анализ ошибок SELinux.

В лабораторной работе используется дистрибутив Rocky Linux 8 с политикой `targeted` и режимом `enforcing`.

4. Выполнение лабораторной работы

4.1 1. Подготовка стенда

Проверено отсутствие пакета `httpd`, затем он установлен командой `sudo yum install -y httpd`. SELinux находится в режиме `enforcing` (`getenforce`), политика `targeted` (`sestatus`). Файрвол отключён для исключения сетевых блокировок. Apache запущен и добавлен в автозагрузку, статус `active (running)` на порту 80 (рис. [Рисунок 1](#)).

Рисунок 1: Подготовка стенда и запуск Apache

4.2 2. Контекст процессов и булевы переключатели

С помощью `ps -eZ | grep httpd` определён контекст процессов `httpd`: `system_u:system_r:httpd_t:s0`. Команда `getsebool -a | grep httpd` показала, что большинство дополнительных возможностей отключены (рис. [Рисунок 2](#)).

```

[aspryadko@aspryadko ~]$ ps -eZ | grep httpd
system_u:system_r:httpd_t:s0      33401 ?        00:00:00 httpd
system_u:system_r:httpd_t:s0      33561 ?        00:00:00 httpd
system_u:system_r:httpd_t:s0      33562 ?        00:00:00 httpd
system_u:system_r:httpd_t:s0      33563 ?        00:00:00 httpd
system_u:system_r:httpd_t:s0      33565 ?        00:00:00 httpd
[aspryadko@aspryadko ~]$ getsebool -a | grep httpd
httpd_anon_write --> off
httpd_built_in_scripting --> on
httpd_can_check_spam --> off
httpd_can_connect_ftp --> off
httpd_can_connect_ldap --> off
httpd_can_connect_mythtv --> off
httpd_can_connect_zabbix --> off
httpd_can_network_connect --> off
httpd_can_network_connect_cobbler --> off
httpd_can_network_connect_db --> off
httpd_can_network_memcache --> off
httpd_can_network_redis --> off
httpd_can_network_relay --> off
httpd_can_sendmail --> off
httpd_dbus_avahi --> off

```

Рисунок 2: Контекст процессов httpd и переключатели

4.3 3. Анализ файловых меток и создание тестового файла

Установлены пакеты `setools-console` и `polyscoreutils-python-utils`. Команда `seinfo` вывела общую статистику политики (типы, пользователи, роли и т.д.).

Просмотр меток каталогов: `/var/www/html` имеет тип `httpd_sys_content_t` (`ls -lZ /var/www`). Создан файл `/var/www/html/test.html` с содержимым

`<html><body>test</body></html>`. Проверен его контекст:

`unconfined_u:object_r:httpd_sys_content_t:s0` (рис. Рисунок 3). После этого файл успешно читается через веб-сервер (`curl http://127.0.0.1/test.html`).

```

[aspryadko@aspryadko ~]$ sudo chcon -t samba_share_t /var/www/html/test.html
[aspryadko@aspryadko ~]$ ls -lZ /var/www/html/test.html
-rw-r--r--. 1 root root unconfined_u:object_r:samba_share_t:s0 31 May  2 17:59 /var/www/html/test.html
[aspryadko@aspryadko ~]$ sudo tail -20 /var/log/messages
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: event4
- VirtualBox mouse integration: client bug: event processing lagging behind by
13ms, your system is too slow
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: event4
- VirtualBox mouse integration: WARNING: log rate limit exceeded (5 msgs per 60
min). Discarding future messages.
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: client
bug: timer event4 debounce: scheduled expiry is in the past (-2ms), your system
is too slow
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: client
bug: timer event4 debounce: scheduled expiry is in the past (-10ms), your system
is too slow
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: client
bug: timer event4 debounce short: scheduled expiry is in the past (-23ms), your
system is too slow
May  2 18:00:15 aspryadko org.gnome.Shell.desktop[2096]: libinput error: event2
- AT Translated Set 2 keyboard: client bug: event processing lagging behind by
16ms, your system is too slow

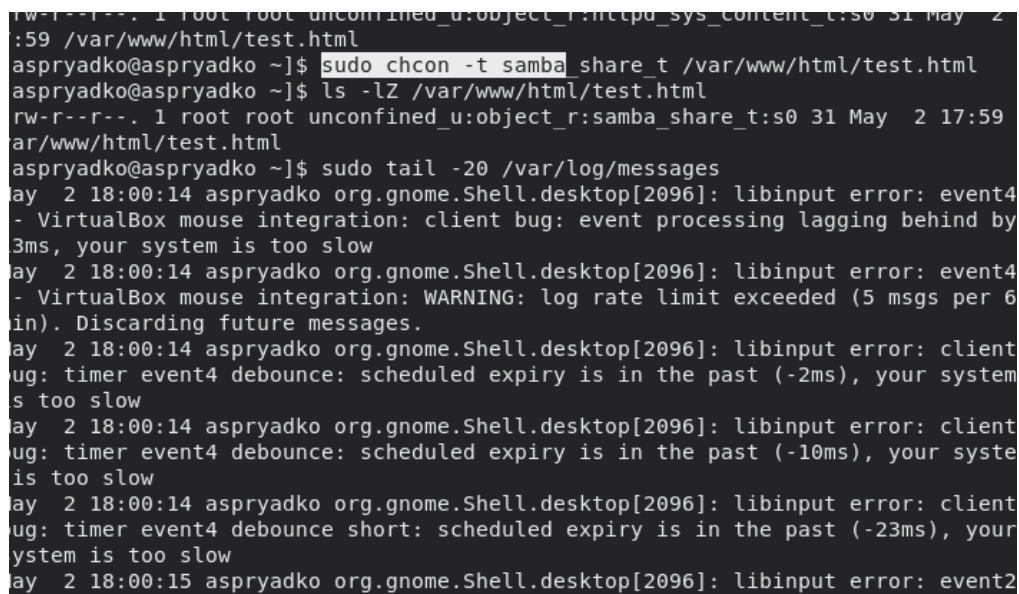
```

Рисунок 3: Метки файлов и создание test.html

4.4 4. Изменение контекста и блокировка доступа

Контекст файла изменён на samba_share_t командой `sudo chcon -t samba_share_t /var/www/html/test.html`. Попытка доступа через curl вызывает ошибку 403 Forbidden (рис. Рисунок 4). Классические права -rw-r--r-- остаются прежними, но доступ запрещён SELinux.

Анализ логов: - В /var/log/httpd/error_log зафиксировано: (13)Permission denied: ... access to /test.html denied. - В /var/log/audit/audit.log появились записи `avc: denied { getattr } ... scontext=...httpd_t ... tcontext=...samba_share_t`. - В /var/log/messages утилита `setroubleshoot` сообщила о блокировке и предложила решения (восстановить метку через `restorecon` или сменить контекст на `public_content_t`).



```
aspryadko@aspryadko ~]$ sudo chcon -t samba_share_t /var/www/html/test.html
aspryadko@aspryadko ~]$ ls -lZ /var/www/html/test.html
-rw-r--r--. 1 root root unconfined_u:object_r:samba_share_t:s0 31 May  2 17:59
/var/www/html/test.html
aspryadko@aspryadko ~]$ sudo tail -20 /var/log/messages
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: event4
- VirtualBox mouse integration: client bug: event processing lagging behind by
3ms, your system is too slow
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: event4
- VirtualBox mouse integration: WARNING: log rate limit exceeded (5 msgs per 6
min). Discarding future messages.
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: client
bug: timer event4 debounce: scheduled expiry is in the past (-2ms), your system
is too slow
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: client
bug: timer event4 debounce: scheduled expiry is in the past (-10ms), your syste
is too slow
May  2 18:00:14 aspryadko org.gnome.Shell.desktop[2096]: libinput error: client
bug: timer event4 debounce short: scheduled expiry is in the past (-23ms), your
system is too slow
May  2 18:00:15 aspryadko org.gnome.Shell.desktop[2096]: libinput error: event2
```

Рисунок 4: Блокировка доступа при неверном контексте и логи

4.5 5. Восстановление доступа

Правильный контекст возвращён командой `sudo chcon -t httpd_sys_content_t /var/www/html/test.html`. Повторный запрос `curl http://127.0.0.1/test.html` снова отдаёт содержимое (рис. Рисунок 5).

```

143,fd=4),("httpd",pid=43142,fd=4),("httpd",pid=43141,fd=4),("httpd",pid=43128,
d=4))
[aspryadko@aspryadko ~]$ sudo chcon -t httpd_sys_content_t /var/www/html/test.html
[aspryadko@aspryadko ~]$ curl http://127.0.0.1:81/test.html
<html><body>test</body></html>
[aspryadko@aspryadko ~]$ sudo nano /etc/httpd/conf/httpd.conf
[aspryadko@aspryadko ~]$ sudo systemctl restart httpd

[aspryadko@aspryadko ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor prese
   Active: reloading (reload) since Sat 2026-05-02 18:05:02 MSK; 55ms ago
     Docs: man:httpd.service(8)
  Main PID: 43527 (httpd)
    Status: "Reading configuration..."
     Tasks: 1 (limit: 24730)
    Memory: 3.0M
    CGroup: /system.slice/httpd.service
            └─43527 /usr/sbin/httpd -DFOREGROUND

May 02 18:05:01 aspryadko.localdomain systemd[1]: httpd.service: Succeeded.
May 02 18:05:01 aspryadko.localdomain systemd[1]: Stopped The Apache HTTP Serve

```

Рисунок 5: Возврат контекста и успешный доступ

4.6 6. Перенос Арасче на порт 81 и SELinux

В файле `/etc/httpd/conf/httpd.conf` строка `Listen 80` заменена на `Listen 81`. После перезапуска Арасче успешно стартует и слушает порт 81, что подтверждено `sudo systemctl status httpd` и `sudo ss -tlnp`. При этом SELinux уже разрешал порт 81 для `http_port_t` (проверено через `sudo semanage port -l | grep http_port_t`), поэтому дополнительная команда `semanage port -a` не потребовалась. Проверка через `curl http://127.0.0.1:81/test.html` показала успешный отклик (рис. Рисунок 6).

```

[aspryadko@aspryadko ~]$ sudo semanage port -l | grep http_port_t
http_port_t                tcp      81, 80, 81, 443, 488, 8008, 8009, 8443,
9000
pegasus_http_port_t         tcp      5988
[aspryadko@aspryadko ~]$ sudo systemctl start httpd
[aspryadko@aspryadko ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor prese
   Active: active (running) since Sat 2026-05-02 18:02:48 MSK; 1min 10s ago
     Docs: man:httpd.service(8)
  Main PID: 43128 (httpd)
    Status: "Running, listening on: port 81"
     Tasks: 213 (limit: 24730)
    Memory: 29.3M
    CGroup: /system.slice/httpd.service
            └─43128 /usr/sbin/httpd -DFOREGROUND
              └─43140 /usr/sbin/httpd -DFOREGROUND
                └─43141 /usr/sbin/httpd -DFOREGROUND
                  └─43142 /usr/sbin/httpd -DFOREGROUND
                    └─43143 /usr/sbin/httpd -DFOREGROUND

```

Рисунок 6: Работа Арасче на порту 81

4.7 7. Возврат к стандартным настройкам и очистка

Возвращена настройка `Listen 80` в конфигурации Apache. Сервер перезапущен, после чего `sudo ss -tlnp` показывает прослушивание порта 80. Порт 81 удалён из политики SELinux командой `sudo semanage port -d -t http_port_t -p tcp 81`. Тестовый файл удалён: `sudo rm /var/www/html/test.html` (рис. Рисунок 7).

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor prese
   Active: reloading (reload) since Sat 2026-05-02 18:05:02 MSK; 55ms ago
     Docs: man:httpd.service(8)
  Main PID: 43527 (httpd)
    Status: "Reading configuration..."
     Tasks: 1 (limit: 24730)
    Memory: 3.0M
    CGroup: /system.slice/httpd.service
            └─43527 /usr/sbin/httpd -DFOREGROUND

May 02 18:05:01 aspryadko.localdomain systemd[1]: httpd.service: Succeeded.
May 02 18:05:01 aspryadko.localdomain systemd[1]: Stopped The Apache HTTP Serve
May 02 18:05:01 aspryadko.localdomain systemd[1]: Starting The Apache HTTP Serve
May 02 18:05:02 aspryadko.localdomain systemd[1]: Started The Apache HTTP Serve

[aspryadko@aspryadko ~]$ sudo ss -tlnp | grep httpd
LISTEN 0      511          *:80          *:~          users:((("httpd",pid=43
541,fd=4),("httpd",pid=43540,fd=4),("httpd",pid=43539,fd=4),("httpd",pid=43527,f
d=4))
[aspryadko@aspryadko ~]$ sudo semanage port -d -t http_port_t -p tcp 81
[aspryadko@aspryadko ~]$ sudo rm /var/www/html/test.html
```

Рисунок 7: Возврат к порту 80 и очистка

5. Выводы

- SELinux в режиме enforcing с политикой targeted контролирует доступ процессов к файлам и сетевым портам на основе контекстов безопасности, а не только классических прав доступа.
- Изменение типа файла с `httpd_sys_content_t` на `samba_share_t` привело к блокировке чтения веб-сервером, что подтверждено логами аудита.
- Возврат правильного контекста восстановил доступ без изменения прав `gwx`.
- Управление портами через `semanage port` позволяет гибко настраивать разрешённые сетевые ресурсы для доменов SELinux.
- Получены практические навыки работы с утилитами `chcon`, `restorecon`, `semanage`, `getsebool`, `sealert`, анализа логов и диагностики ошибок SELinux.

Список литературы

1. Методические указания к лабораторной работе №6 «Мандатное разграничение прав в Linux (SELinux)».

...